Dynamic Programming



Last lecture!

Review – Lecture 12

Dynamic Programming - a general technique for solving problems that have a nice sub-problems structure.

Recursion may lead to an exponential running time, since the same sub-problem may be solved many times.

The idea: Solve every sub-problem only once, keep the results in a table.

Review – Lecture 12

Solving a problem by DP consists of the following steps:

1. Define the table (decide what are the sub-problems).
2. Identify base cases (sub-problems that can be solved directly).
3. Write a recursive formula, and then, instead of performing the recursion, fill the table, starting from the base cases.
4. Conclude the value of the solution from the table.
5. (optional) Retrieve the solution.

Review – Lecture 12

Examples:

1. Fibonacci numbers
2. Max-sum subsequence
3. Most profitable tour
4. Longest common subsequence
5. All-pair shortest path

Coin Change



- We are given an infinite supply of n types of coins having integer values $c_1, \ldots, c_n$.
- Given an integer number B, suggest a dynamic programming algorithm to determine the minimal number of coins that can be used to return the change value of B. In case no such change exists, the algorithm should detect it.
- Yes, for some sets of coins, this problem can be solved by a greedy algorithm that always take the max-value coin that fits in. For other sets – greedy fails.

Coin Change



- Define the table: Let $M[j]$ be the minimal number of coins required to change value j. If such a change does not exist, then $M[j] = \infty$.
- Base case: For $j < \min(c_1, \ldots, c_n)$, clearly $M[j] = \infty$.

  Except for $j = 0$, for which $M[0] = 0$.

  (Instead of $M[0] = 0$, we could set $M[c_k] = 1$, for all $1 \leq k \leq n$)
- The output lies at $M[B]$.

Coin Change



- General case:

  $M[j] = 1 + min_{1 \le k \le n}\{M[j - c_k]\}$.

  That is, we consider all the possibilities for the last coin.

- M is a 1-dim table, Filling M is done from $j = 0$ to $j = B$.
- Time complexity: Each entry takes $O(n)$. The total time complexity is $O(Bn)$.

Coin Change



- Example: $c_1 = 3, c_2 = 5, c_2 = 7$

  $B = 17$

	-x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
M	∞	0	∞	∞	1	∞	1	2	1	2	3	2	3	2	3	2	3	4	3

Coin Change



- Example: $c_1 = 3, c_2 = 5, c_2 = 7$

  $B = 17$

	-x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
M	∞	0	∞	∞	1	∞	1	2	1	2	3	2	3	2	3	2	3	4	3
Last					3		5	3	7	3	3	3	5	5	3	7	3	3	3

If we want to retrieve the actual set of coins – keep another entry, Last, with the value of the coin determining $M[j]$.



Rented Cars Problem

You need to pass a long way in rented cars. There are $n+1$ car rental agencies along the way $(0,1,\ldots,n)$.

For every $0 \le i < j \le n$, the cost of renting a car in agency i and returning it in agency j is known, and denoted $p(i,j)$.

It is impossible to drive backward, and you cannot assume that the costs depend on the way (in particular, it is possible for example that $p(1,8) < p(1,3)$).

Rented Cars Problem



1. Suggest an algorithm, based on dynamic programming for calculating the minimal cost of driving from agency 0 to agency n.
2. Suggest an algorithm, based on dynamic programming for calculating the minimal cost of driving from agency 0 to agency n, assuming you are allowed at most B car exchanges.
3. Show that each of the following greedy algorithms is not optimal:
   - In agency i, take a car to agency j for which p(i,j) is minimal.
   - In agency i, take a car to agency j for which p(i,j)/(j-i) is minimal.

Rented Cars Problem



1. Suggest an algorithm, based on dynamic programming for calculating the minimal cost of driving from agency 0 to agency n.

Solution: Denote by $C[j]$ the minimal cost of traveling from agency 0 to agency j.

Base: $C[0] = 0$. Output: $C[n]$

DP formulation: $C[j] = min_{1 \le k < j}\{C[k] + p(k,j)\}$.

Explanation: For every agency $1 \le k < j$, we check the cost of a travel in which the last exchange takes place in agency k.

Complexity: O(j) for entry j. Total: $O(1+2+\ldots+n)=O(n^2)$

Rented Cars Problem

2. Suggest an algorithm, based on dynamic programming for calculating the minimal cost of driving from agency 0 to agency n, assuming you are allowed at most B car exchanges.

Solution: Denote by $C[s,j]$ the minimal cost of traveling from agency 0 to agency j with exactly s exchanges.

$C[s,j]$ is well-define for $s < j$. For$s \geq j$ we can either initialize $C[s,j] = M$ [for $M > p(0,n)$], or avoid reference to these entries.

Base case: $s = 0$. $C[0,0] = 0$, and for every $j > 0$, $C[0,j] = p(0,j)$.
Output: min $_{0 \leq s \leq B}$ $C[s,n]$.

Rented Cars Problem

Recursive formula: For all $0 < s < j$,

Remark: the range of k can be 0<k<j if C was initialized correctly for s≥j)

$$C[s,j] = min_{s-1 \le k < j}\{C[s-1,k] + p(k,j)\}.$$

Explanation: For every agency 0≤k<j, we check the cost of a travel in which the s-th exchange takes place in agency k. This means that in the way from 0 to k there were s-1 exchanges.

Time complexity: There are n columns and B rows.
The table is filled row after row.
Each row takes $O(n^2)$, for a total of $O(n^2B)$.

Rented Cars Problem



3. Show that each of the following greedy algorithms is not optimal:
   - In agency i, take a car to agency j for which p(i,j) is minimal.
     - p(0,1)=2, p(1,2)=8, p(0,2)=3
   - In agency i, take a car to agency j for which p(i,j)/(j-i) is minimal.
     - p(0,1)=2, p(1,2)=8, p(0,2)=6

Subset Sum

Given an array of n positive integers $A=\{a_1, a_2, \ldots, a_n\}$, and an integer M.
Suggest an algorithm for deciding if A has a non-empty subset D such that M divides their sum.

Example: A = {8, 7, 13, 8, 5, 17}. M=11.
Yes, for D={8,8,17}, because (8+8+17)= 33 = 11*3.
Also for D={5,17}, because (5+17)=22=11*2.

Subset Sum

Note that the problem is equivalent to asking whether there is a non-empty subset D of A such that $\sum_{i \epsilon D} a_i \; mod \; M = 0$?

DP table definition: For every $1 \leq j \leq n$, $0 \leq r \leq M-1$, let B[j,r] be a Boolean value such that B[j,r]=True iff $\{a_1, a_2, \ldots, a_j\}$ has a non-empty subset whose sum modulo M is r.

Base Case: for all $0 \leq r \leq M-1$, B[1,r]=True iff $a_1 \bmod M = r$. (otherwise, B[1,r]=False)

Our output: B[n,0].

Subset Sum

DP formulation:

$$B[j,r] = (a_j \bmod M = r) \text{ OR } B[j-1,r] \text{ OR } B[j-1, (r-a_j) \bmod M]$$

Explanation: $\{a_1, a_2, \ldots, a_j\}$ has a non-empty subset whose value mod M is r, if at least one of the following holds:

1. a_j is the only member of the subset.
2. a_j is not in the subset, and then $\{a_1, a_2, \ldots, a_{j-1}\}$ has a subset whose sum mod M is r.
3. a_j is in the subset, and then $\{a_1, a_2, \ldots, a_{j-1}\}$ has a subset whose sum mod M equals $(r-a_j)$ mod M.

Subset Sum

The table has n rows and M columns.

Fill the table :
Top row (j=1) is the base case.
Then fill row after row (for j= 2 to n-1, r = 0 to M-1).
Then fill (and output) B[n,0].

Remark: If B[j,0] = True at some stage, it is OK to halt and announce a positive answer.

Every entry depends on two entries from the previous row.
Complexity: O(1) per entry. Total: O(Mn)

Subset Sum

(Partial) Example: A = {8, 7, 13, 8, 5, 17}. M=11

j	a_j	0	1	2	3	4	5	6	7	8	9	10
1	8	F	F	F	F	F	F	F	F	T	F	F
2	7	F	F	F	F	T	F	F	T	T	F	F
3	13	F	F	T	F	T	F	T	T	T	T	T
4	8											
5	5											
6	17											

Dora and Boots Coin Game



Dora and Boots are playing the following game:

n piles of coins are placed in a row on a table. The i-th pile has a_i coins

Each player, in turn, takes one of the extreme piles (leftmost or rightmost).

The goal is to collect the maximal number of coins.



Dora and Boots Coin Game



1. Consider the following greedy algorithm: Take the highest pile between the leftmost and rightmost.

Is it optimal?



2. Suggest an algorithm, based on dynamic programming for calculating the maximal revenue of Dora if she is the first player.

Dora and Boots Coin Game



1. Consider the following greedy algorithm: Take the highest pile between the leftmost and rightmost.

Is it optimal?



Answer: No : For example 1,1,8,2.

If Dora begins and takes the '2', then Boots will take 8+1=9. If Dora takes the '1', then Boots will have to reveal the '8'.

Dora and Boots Coin Game



2. Suggest an algorithm, based on dynamic programming for calculating the maximal revenue of Dora if she is the first player.



Dora and Boots Coin Game



2. Suggest an algorithm, based on dynamic programming for calculating the maximal revenue of Dora if she is the first player.



Preprocessing: calculate $S_{ij} = a_i + \cdots + a_j$ for all $1 \le i \le j \le n$.

In the example [1,1,8,2], S_{24}=11, S_{33}=8.

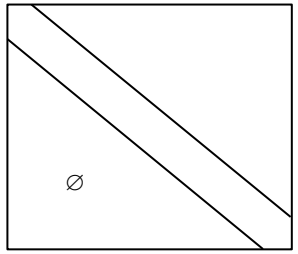
Dora and Boots Coin Game



$S_{ij} = a_i + \cdots + a_j$ for all $1 \le i \le j \le n$

The Table: Let T[i,j] be the maximal revenue from piles i,..,j for the player whose turn is next.
T[i,j] is defined for all pairs $1 \le i \le j \le n$
The output: T[1,n].
Base case: for all $1 \le i \le n$, $T[i,i]=a_i$





Dora and Boots Coin Game

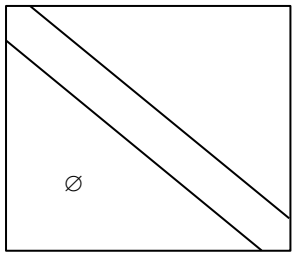


Recursive formula:

$$T[i,j] = \max\{a_i + \left(S_{i+1,j} - T[i+1,j]\right),$$

$$\underline{\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad} \}$$





Dora and Boots Coin Game

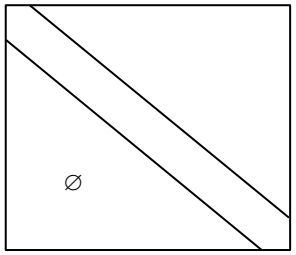




Recursive formula:

$$T[i,j] = \max\{a_i + \left(S_{i+1,j} - T[i+1,j]\right),$$
$$a_j + S_{i,j-1} - T[i,j-1])\}$$

Explanation: If the player takes a_i, then the other player will have revenue T[i+1,j] out of the remaining range $a_{i+1}…a_j$.
If the player takes a_j, then the other player will have revenue T[i,j-1] out of the remaining range $a_i…a_{j-1}$.

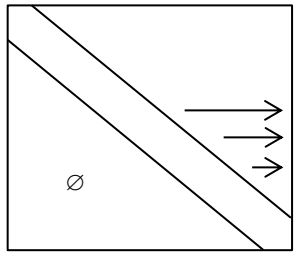


Dora and Boots Coin Game



Filling the table: The base case refers to the main diagonal.
Each entry depends on the entry below it and the entry left to it. Therefore, we can advance row after row from the bottom, filling every entry for which $1 \le i \le j \le n$.





Complexity:
preprocessing (calculating S): $O(n^2)$.
Filling the table: O(1) for entry i,j. Total: $O(n^2)$


That's all Folks!